

CSC310 Data Structures and Algorithm Analysis (Dr. Ng)

Programming Assignment: Task List (ARRAY)

Write a program to keep track of a task list (up to maximum 8) using a simple menu system. Your program should be able to adapt to other maximum value, such as 10, 20, 60, or 100, with minimal code changes.

In this particular implementation, you are going to use **ARRAY** as your internal data structure. NO pointers or linked list structure is allowed for this assignment.

Note the first task of the list is indexed from 1.

Your program should provide the user with a menu system:

The main menu provides 6 choices options:

1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit

Choice →

Detailed description of each option:

- 1: Program prompts the user for the **task name** and the **position of the task after addition**. Task name is composed of exactly 2 alphabets without space.

Even though it is not the best User-friendly interface but to keep this assignment simple, there is NO default position mechanism. Both the task name and its position p MUST be entered by the user on a single line.

If p is specified and valid, then the task is inserted to be the pth task in the list.

Enter task name and its position -> **AB 1**

Number of tasks = 1

1:AB

1:AB

Enter task name and its position -> **CD 1**

Number of tasks = 2

1:CD 2:AB

2:AB 1:CD

Enter task name and its position -> EF 2

```
Number of tasks = 3
1:CD 2:EF 3:AB
3:AB 2:EF 3:CD
```

Enter task name and its position -> GH 4

```
Number of tasks = 4
1:CD 2:EF 3:AB 4:GH
4:GH 3:AB 2:EF 1:CD
```

You need to check if the specified position p is valid with respect to the addition operation.

Task is added one at a time. Each time a task is added, **the program automatically prints the number of tasks in the task list and displays the task list (with position information) in both order and reverse order, and then returns to the main menu again.**

- 2: Program prompts the user for the position q of the task to be deleted and print out the task name that has been deleted.

Even though it is not the best User-friendly interface but to keep this assignment simple, there is NO default position mechanism. The position q of the task to be deleted must be entered by the user.

```
// Delete the first task in the list
// Delete the second task in the list
```

Enter its position -> 1

```
Number of tasks = 3
1:EF 2:AB 3:GH
3:GH 2:AB 1:EF
```

Enter its position -> 2

```
Number of tasks = 2
1:EF 2:GH
2:GH 1:EF
```

You need to check if the specified position q is valid with respect to the delete operation.

Task is deleted one at a time. Each time a task is deleted, **the program automatically prints the number of tasks in the task list and displays the task list in both order and reverse order, and then returns to the main menu again.**

- 3: Program prints out the number of tasks in the list and returns back to the main menu
- 4: Program prints out the list of tasks by its name, one-by-one in a single row/horizontal format, in the order of the indices of the tasks, and returns back to the main menu
- 5: Program prints out the list of tasks by its name, one-by-one in a single row/horizontal format, in the REVERSE order of the indices of the tasks and returns back to the main menu. Note that you are not using any extra data structures.
- 6: Program exits the menu and terminates.

Please test your program thoroughly and make sure it is free of errors. To simplify this assignment, **you need to check if the given input is correct with respect to the respective context, highlighted in yellow.** Points will be deducted for each error that I discovered for you.

Please also pay attention to program design and follow all the good programming practice. I reserve the right to deduct points based on these criteria.

Submit: Program design
Hierarchical diagrams/ Structure charts
Procedure specification
Program listing
Testing cases with Justification
Results of test cases